



National University of Computer and Emerging Sciences

Generative Adversarial Networks

AI-4009 Generative AI

Dr. Akhtar Jamil

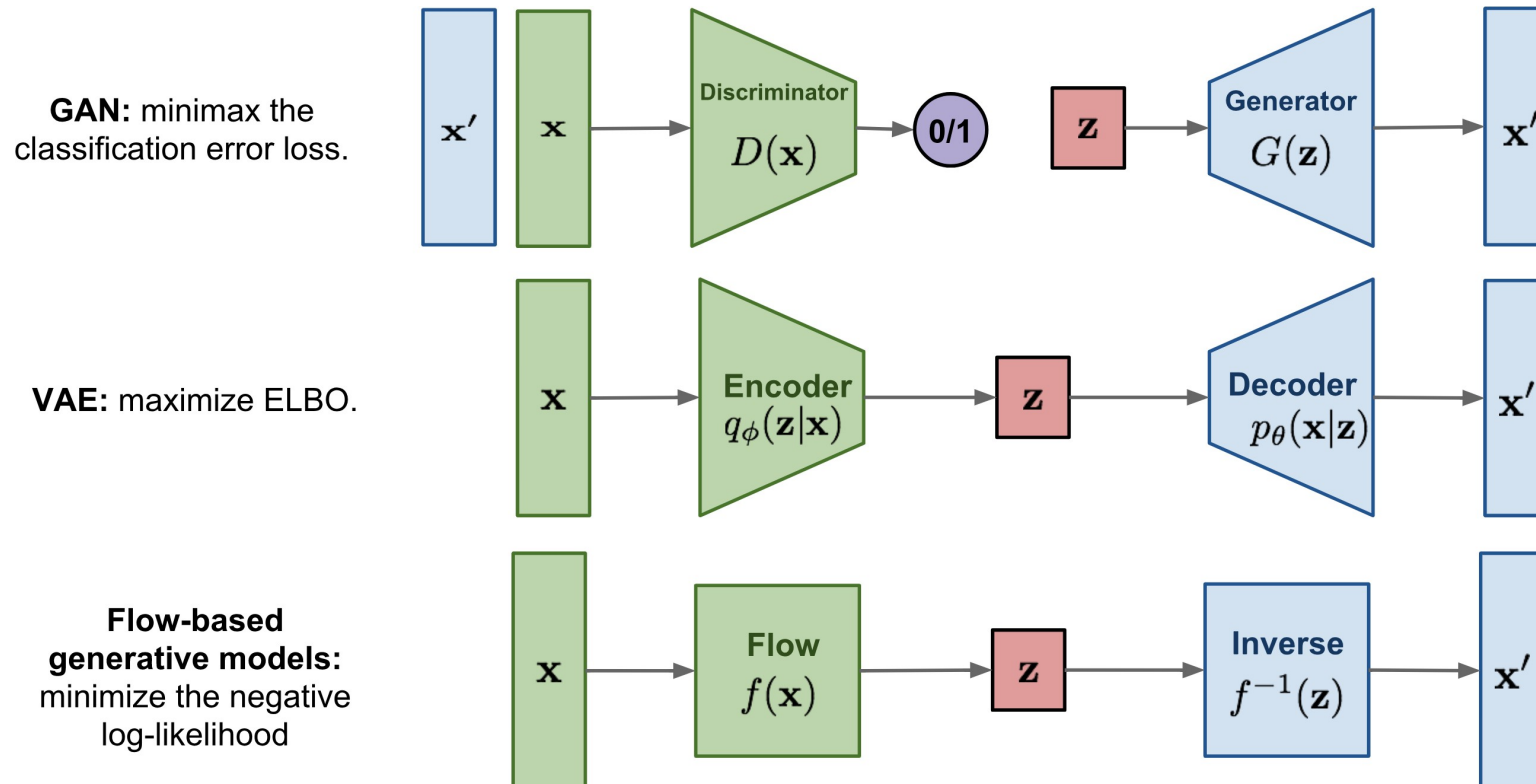
Department of Computer Science

Goals

- Review of Previous Lecture
- Today's Lecture
 - Problems Training GANs
 - DCGANs

Review of Previous Lecture

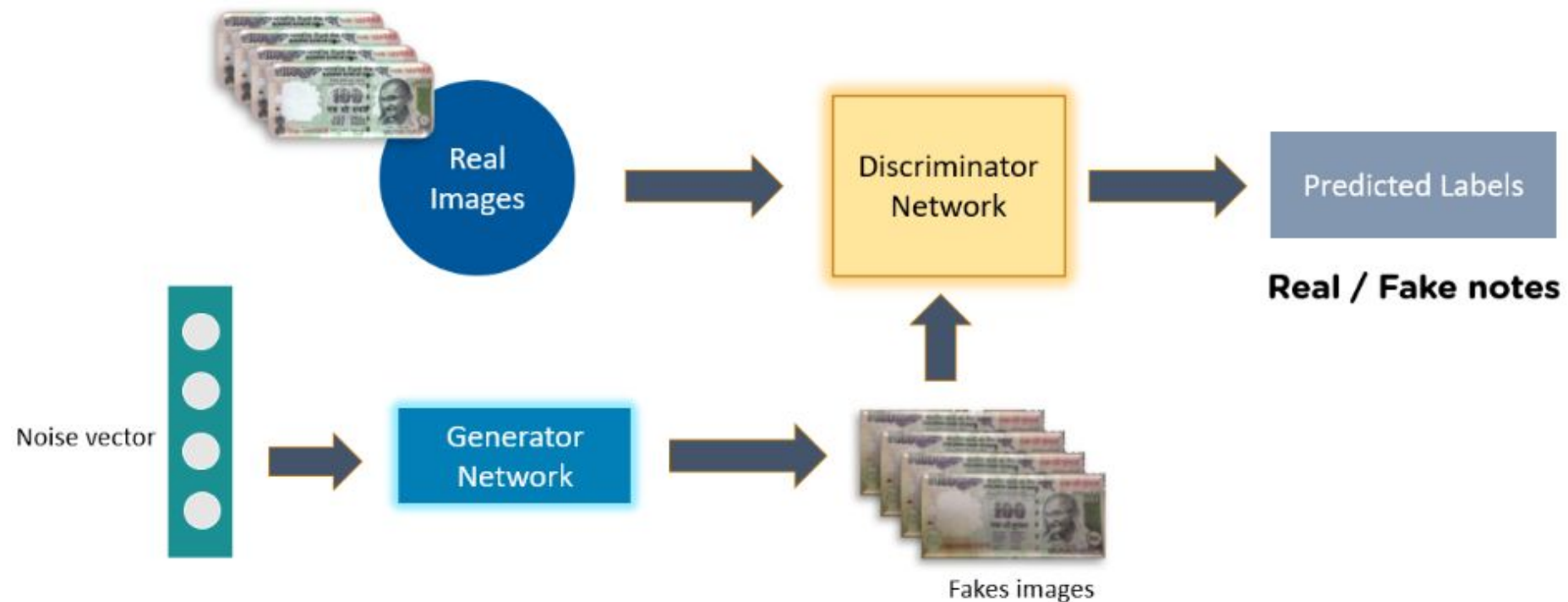
Types of Generative Models



Lilian Weng, "[Flow-based Deep Generative Models](#)," *blog post*, 2018.

Generative Adversarial Network

- Complete GAN Network

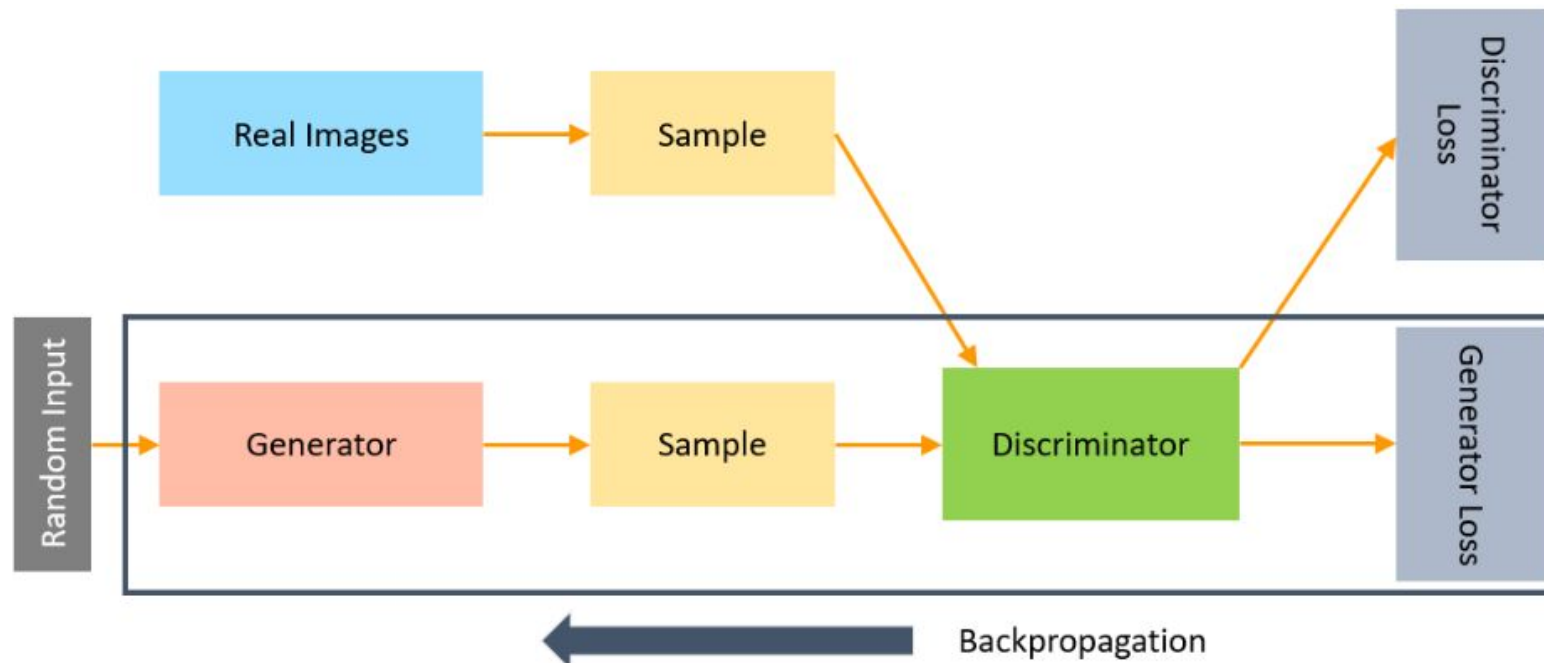


GAN Training

- Because a GAN contains two separately trained networks, its training algorithm must address two complications:
 - GANs must switch two different kinds of training (generator and discriminator).
 - GAN convergence is hard to identify.
- **Alternating Training**
 1. The discriminator trains for one or more epochs.
 2. The generator trains for one or more epochs.
 3. Repeat steps 1 and 2 to continue to train the generator and discriminator networks.

Generative Adversarial Network

- The **backpropagation** method is used to adjust each weight in the right direction by calculating the weight's impact on the output.



Loss Functions

- **Minimax Loss**

- Approach based on game theory
- Two networks:
 1. Generator $g(\mathbf{z}; \mathbf{W}_g) \rightarrow \mathbf{x}$
 2. Discriminator $d(\mathbf{x}; \mathbf{W}_d) \rightarrow \Pr(\mathbf{x} \text{ is real})$

- Objective

$$\begin{aligned} \min_{\mathbf{W}_g} \max_{\mathbf{W}_d} \sum_n \log \Pr(\mathbf{x}_n \text{ is real}; \mathbf{W}_d) + \log \Pr(g(\mathbf{z}_n; \mathbf{W}_g) \text{ is fake}; \mathbf{W}_d) \\ \equiv \min_{\mathbf{W}_g} \max_{\mathbf{W}_d} \sum_n \log d(\mathbf{x}_n; \mathbf{W}_d) + \log (1 - d(g(\mathbf{z}_n; \mathbf{W}_g); \mathbf{W}_d)) \end{aligned}$$

Today's Lecture

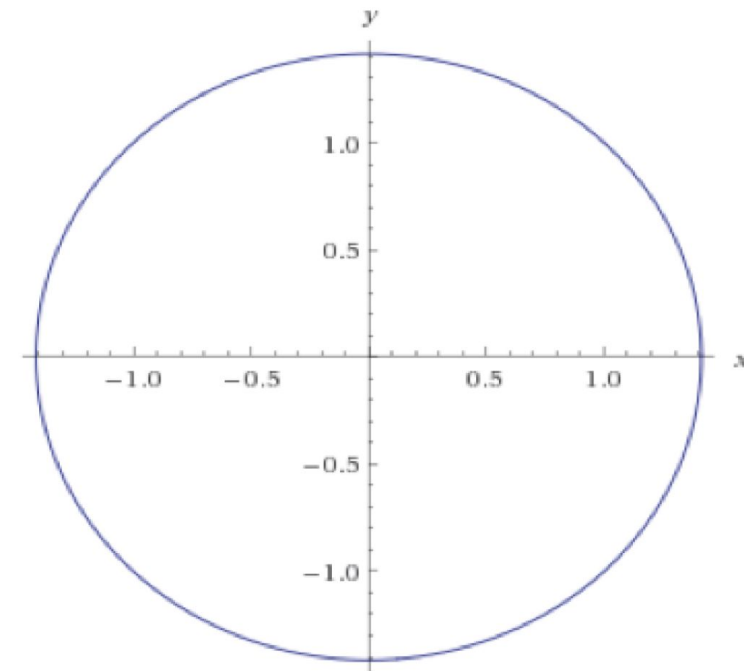
Training Problems

- **Non-Convergence**
- Mode-Collapse

Non-Convergence

$$\min_x \max_y xy$$

- $\frac{\partial}{\partial x} = -y \quad \dots \quad \frac{\partial}{\partial y} = x$
- $\frac{\partial^2}{\partial y^2} = \frac{\partial}{\partial x} = -y$
- Differential equation's solution has sinusoidal terms
- Even with a small learning rate, it will not converge

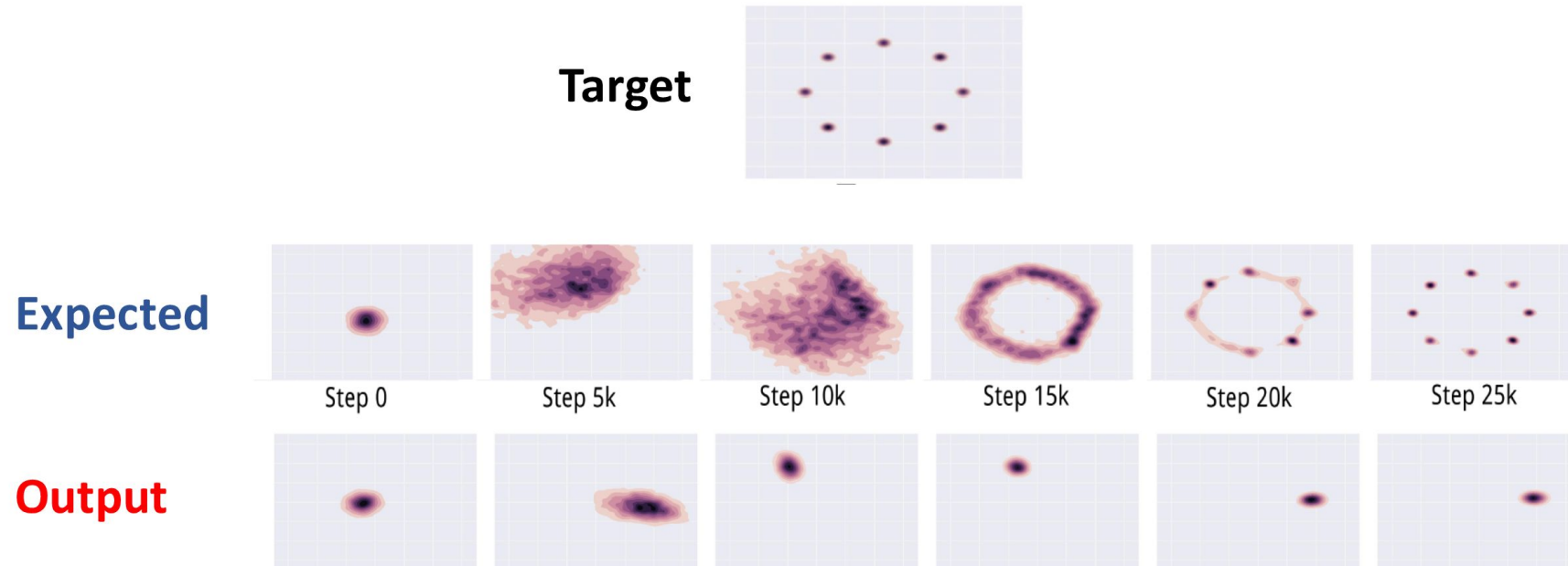


Problems with GANs

- Non-Convergence
- **Mode-Collapse**

Mode-Collapse

- Generator fails to output diverse samples



Metz, Luke, et al. "Unrolled Generative Adversarial Networks." arXiv preprint arXiv:1611.02163 (2016).

Some real examples

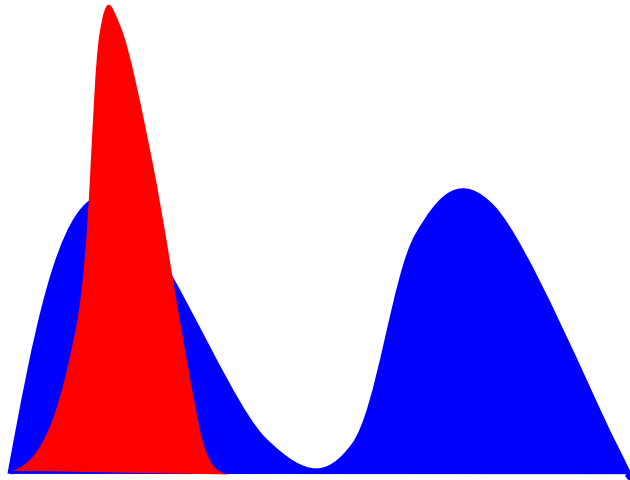


Reed, S., et al. *Generating interpretable images with controllable structure*. Technical report, 2016. 2, 2016.

Mode Collapse

Converge to same faces

Generated
Distribution



Data
Distribution



Sometimes, this is hard to tell since one sees only what's generated, but not what's missed.

Solutions

- Use Mini Batch GANs
- Supervision with Labels

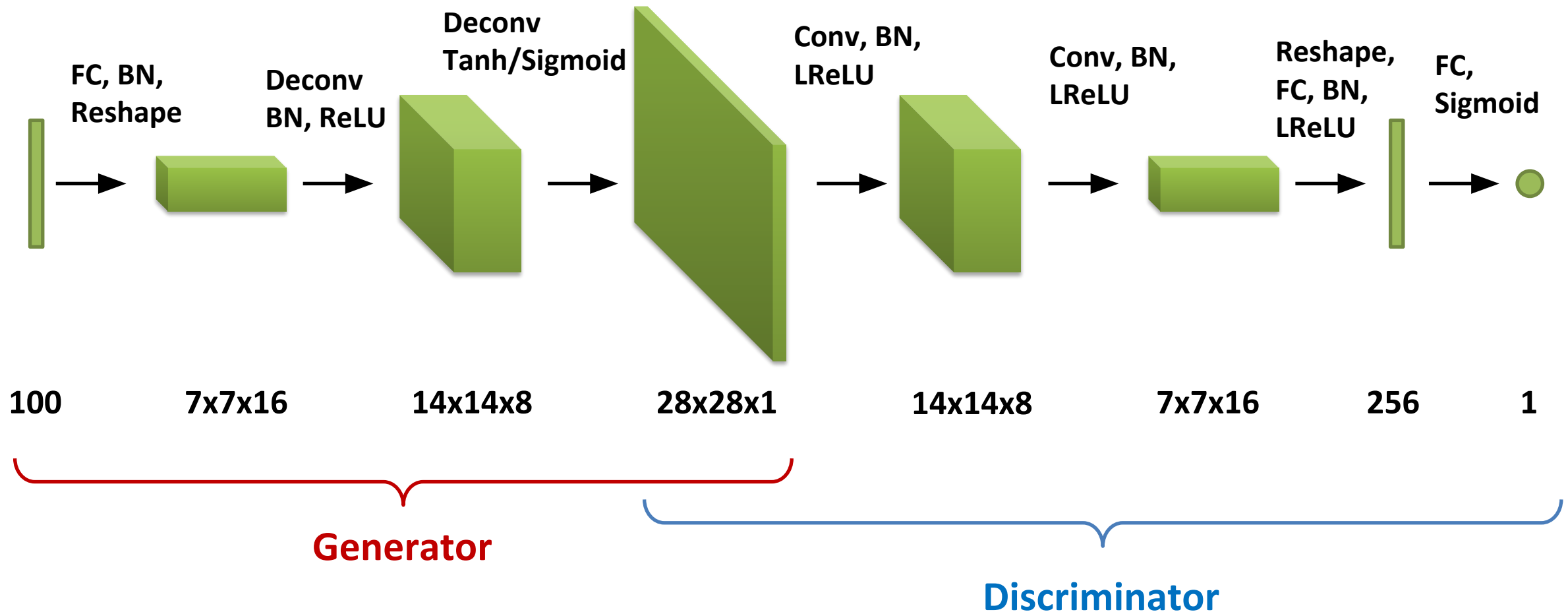
Basic (Heuristic) Solutions

- **Mini-Batch GANs**
- Supervision with labels

Mini Batch Generative Adversarial Networks (Mini Batch GANs)

- **Mini Batch GANs** are a variation of Generative Adversarial Networks (GANs).
- In traditional GANs, the discriminator evaluates each sample individually to decide whether it is real or fake.
- In Mini Batch GANs, the discriminator evaluates samples in small groups or "mini-batches".
 - Concept called mini-batch discrimination
 - Assess the authenticity of individual samples but also the diversity between samples within a batch.

Mini Batch GANs on MNIST



Mini Batch GANs

- **Generator in Batches**

- The generator can **produce batches of images at once** instead of single images.
- Processing in batches allows for more efficient computation.
- More **diversity of samples** being evaluated by the **discriminator**, facilitating a more comprehensive training.

- **Mini-batch Discrimination**

- The **discriminator uses a measure to assess how much the generated samples deviate from the real samples** within the same batch.
- **Computing distances or similarities** between samples in the feature space.
- How diverse the generated images are within each batch (mode collapse).

Mini Batch GANs

- Backpropagation:
 - Based on the discriminator's feedback the GAN updates both the generator and discriminator.
 - The generator learns to produce more diverse and realistic samples
- Iteration:
 - This process repeats over many cycles
 - The generator and discriminator iteratively improving in response to each other's feedback.

Mini Batch GANs

- For standard Discriminator in GAN:

$$L_D = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(D(x_i)) + (1 - y_i) \log(1 - D(G(z_i))) \right]$$

- For Generator:

$$L_G = -\frac{1}{N} \sum_{i=1}^N \log \left(D(G(z_i)) \right)$$

Mini Batch GANs

• Mini-Batch Discrimination

- For each pair of samples (x_i, x_j) in a **mini-batch**, compute a distance measure between features
- Raw data or features learned by earlier layers of the discriminator.
- Distance can be: L1 or L2 or other forms of similarity.
- The mini-batch discrimination value for a single sample (x_i) can be calculated in the batch as:

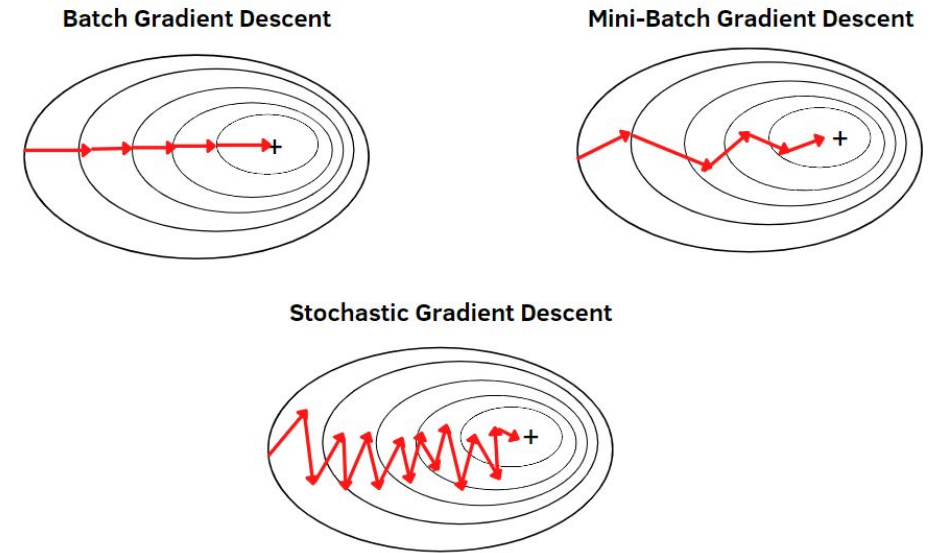
$$T_b(x_i) = \sum_{j=1}^N e^{-||o(x_i) - o(x_j)||}$$

- Modified Discriminator Loss with Mini-Batch Discrimination

$$L_D = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \left(D \left(x_i, T_b(x_i) \right) \right) + (1 - y_i) \log \left(1 - D \left(G(z_i), T_b(G(z_i)) \right) \right) \right]$$

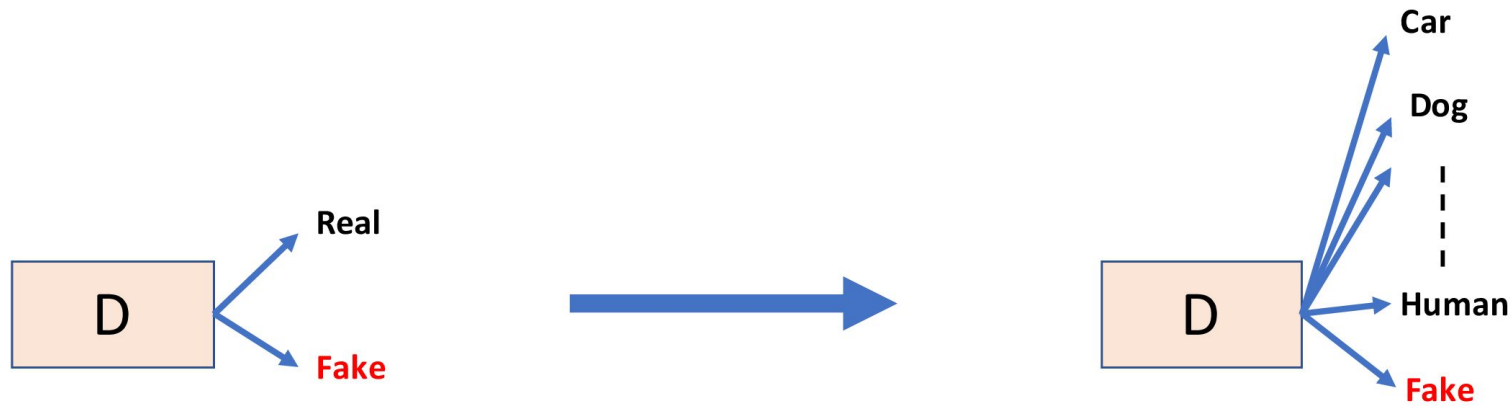
Advantages of Mini Batch GANs

1. Mini Batch GANs encourage the generator to **produce a diverse range of outputs**.
2. **Stable training** as the discriminator works on a batch instead of individual samples.
3. **Mini batches** can lead to **faster convergence of the network**
4. Mini-batch discrimination provides more information to the generator, which can help in generating higher-quality samples.



Supervision with Labels

- Label information of the real data might help



- Empirically generates much better samples

Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks

DCGAN

Unsupervised representation learning with deep convolutional generative adversarial networks

[A Radford](#), [L Metz](#), [S Chintala](#)

arXiv preprint [arXiv:1511.06434](#), 2015 · [arxiv.org](#)

In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications. Comparatively, unsupervised learning with CNNs has received less attention. In this work we hope to help bridge the gap between the success of CNNs for supervised learning and unsupervised learning. We introduce a class of CNNs called deep convolutional generative adversarial networks (DCGANs), that have certain architectural constraints, and demonstrate that they are a strong candidate for

SHOW MORE ▾

☆ Save  Cite Cited by 18550 Related articles All 6 versions Import into BibTeX 

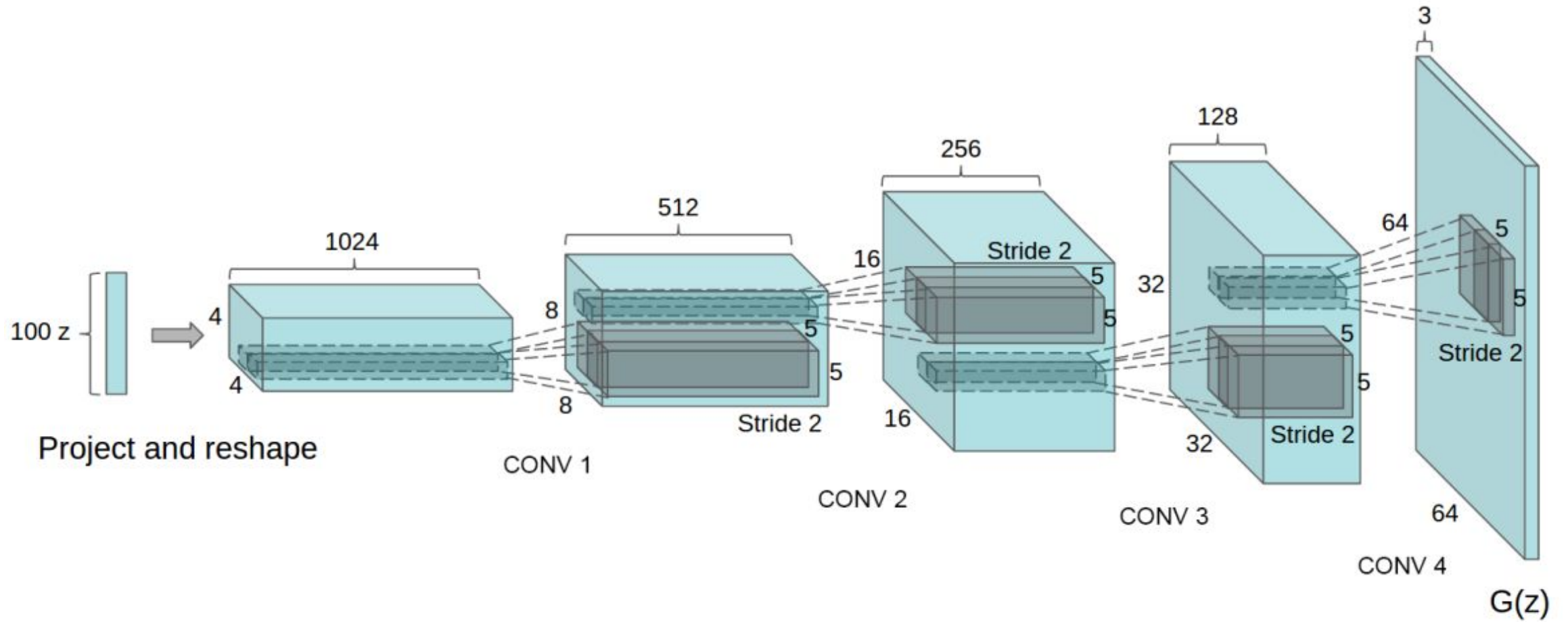
DCGANs

- DCGAN: [Download Paper](#).
- Supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications
- Comparatively, unsupervised learning with CNNs has received less attention
- CNN for unsupervised learning
- DCGANs builds upon the original Generative Adversarial Networks (GANs) by incorporating convolutional layers, making them more suited for image processing

DCGANs

- DCGANs consist of two main components:
- **Generator:** This network generates new data instances (e.g., images) from random noise.
 - It typically uses transposed convolutional layers to progressively upsample the input noise to an image.
- **Discriminator:** This network evaluates the authenticity of images.
 - It takes an image (either from the training set or generated by the generator) and outputs the probability of the image being real (as opposed to being generated).

DCGANs



DETAILS OF ADVERSARIAL TRAINING

- We trained DCGANs on three datasets:
 - Large-scale Scene Understanding (LSUN)
 - Imagenet-1k
 - Faces dataset.
- No pre-processing was applied to training images besides **scaling to the range of the tanh activation function $[-1, 1]$** .
- LeakyReLU with the slope of the leak was set to 0.2.
- Adam optimizer with tuned hyperparameters.
- Learning rate set to 0.0002, momentum = 0.2

Image De-duplication Process

- It is a method used to **decrease the likelihood that GAN memorizes and directly replicates** its training images
- Model learns to understand and recreate the **underlying data distribution without copying exact details**.
- Before deduplication, images from the training set are modified by **cropping them to focus on the central region** and then resizing them to a standard size of **32x32 pixels**.
- **3072-128-3072 de-noising dropout** regularized RELU autoencoder is used
 - Hidden code layer with 128 nodes.

Image De-duplication Process

- Autoencoder compresses and then reconstructs the images, helping to **remove noise and unnecessary details**.
- **Binarization and semantic hashing:**
- After training, the latent spaces are used to represent each image.
- Z are made binary (0 or 1) by thresholding: values above the threshold are set to 1, and those below are set to 0.
- **Result of binarization is like semantic hashing** where similar images are likely to have **similar binary codes**, allowing for efficient comparison and deduplication.

Image De-duplication Process

- **De-duplication**
- Using these binary codes images are compared and deduplicated.
- **Visual inspection and hash collisions**: The authors manually inspected cases where different images had the same binary code (hash collisions).
- The technique effectively identified and removed about **275,000 near-duplicate images from the dataset**

DCGANs

- **Convolutional Layers:** DCGANs use convolutional layers (in the discriminator) and transpose convolutional layers (in the generator)
- **Working Process:**
 - The **generator** creates images from random noise.
 - The **discriminator** assesses these images and real images from the dataset, trying to distinguish between the two.
 - The output of the discriminator is a **probability score** that represents how likely it is that the image is real.

Training of DCGANs

1. Training the Discriminator:

- In each training step, the discriminator is trained first.
- It is provided a batch of real images (labeled as real) and a batch of fake images generated by the generator (labeled as fake).
- The goal is to maximize the probability of correctly classifying both real and fake images.

2. Training the Generator:

- After updating the discriminator, the generator is trained.
- It generates a batch of images, which are then passed to the discriminator.
- The generator's goal is to minimize the likelihood that the discriminator correctly identifies the images as fake (Adversarial).

Training of DCGANs

1. Adversarial Process:

- As training progresses, the generator improves in generating realistic images, the discriminator must also improve at distinguishing fake images from real ones, and vice versa.
- This competition drives both networks to improve (adversarial).

2. Backpropagation and Optimization:

- Both networks use backpropagation to update their weights.
- This iterative process of alternating between training the discriminator and the generator continues until a stopping criterion is met (like a fixed number of epochs or a desired level of performance).

3. Loss Functions:

- Commonly used loss functions include binary cross-entropy
- The generator's loss is based on how well it tricks the discriminator.

Key Features

- **Stability:**

- DCGANs often offer more stable training compared to traditional GANs, partly due to their convolutional nature.

- **Hyperparameters:**

- Tuning hyperparameters (like learning rates, batch size, etc.) is crucial for effective training.

- **Mode Collapse:**

- A common issue with GANs, including DCGANs, is **mode collapse**, where the generator produces limited varieties of outputs.

Key challenges

- Some strategies to mitigate **Mode collapse**:
- **Adding Noise to Inputs:**
 - Introducing noise to the inputs of the discriminator can prevent the discriminator from becoming too confident.
 - This uncertainty can prevent the generator from exploiting weaknesses in the discriminator, leading to a more stable and diverse output.
- **Regularization Techniques:**
 - Applying regularization methods like gradient penalty or weight normalization can help in stabilizing the training and thus prevent mode collapse.
- **Using Different Architectures or Loss Functions:**
 - Sometimes, simply changing the architecture of the GAN or using a different loss function can mitigate mode collapse.
- **Conditional GANs**

Regularization

1. Cost Function with L1 Regularization (Lasso):

$$J(W) = \frac{1}{2m} \sum_{i=1}^m (h_W(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |W_j|$$

2. Cost Function with L2 Regularization (Ridge):

$$J(W) = \frac{1}{2m} \sum_{i=1}^m (h_W(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n W_j^2$$

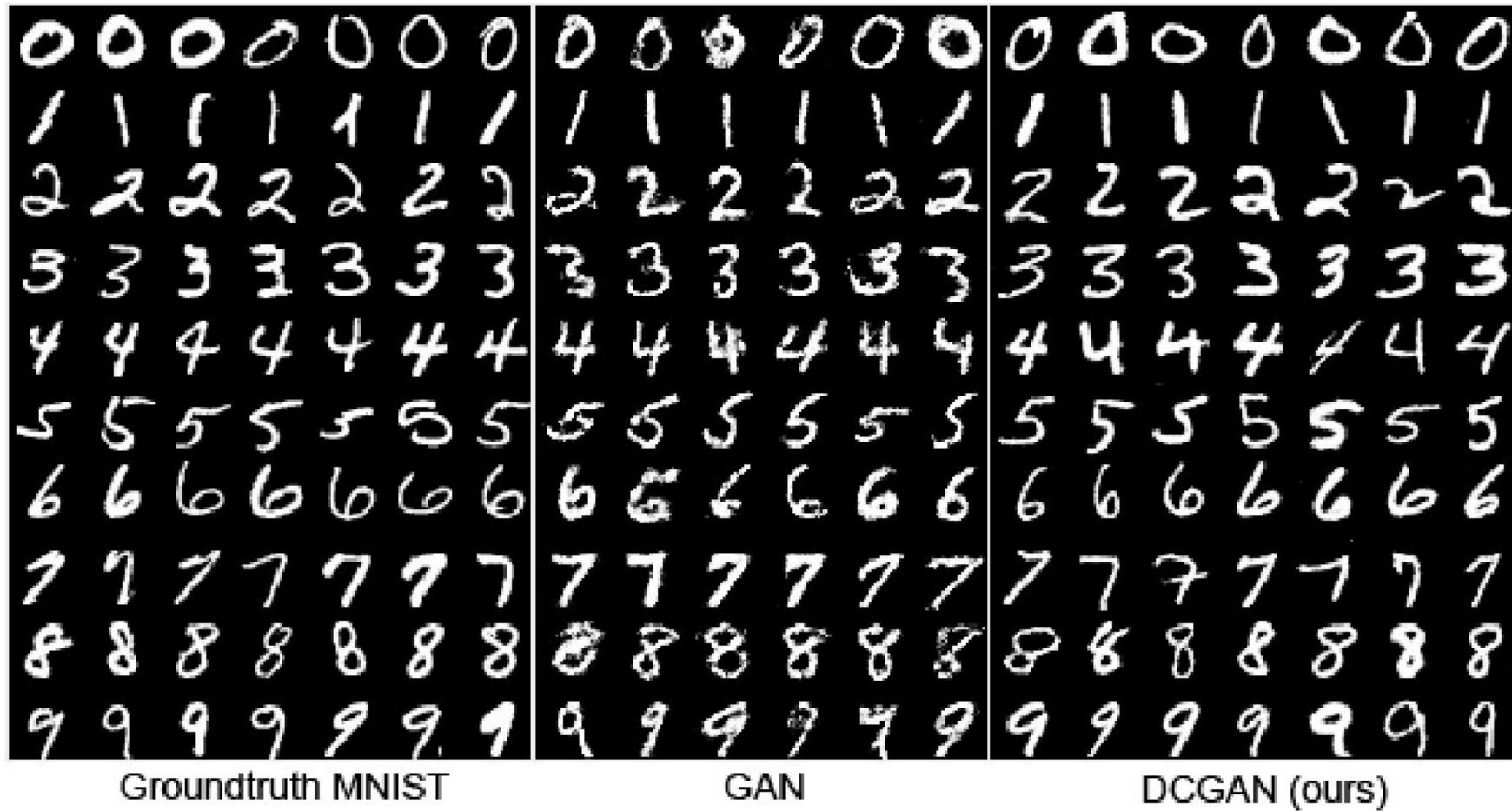
3. Cost Function with Combined L1 and L2 Regularization (Elastic Net):

$$J(W) = \frac{1}{2m} \sum_{i=1}^m (h_W(x^{(i)}) - y^{(i)})^2 + \lambda_1 \sum_{j=1}^n |W_j| + \lambda_2 \sum_{j=1}^n W_j^2$$

Regularization

- λ is the regularization strength hyperparameter that controls the amount of regularization applied.
- Choosing the right value of λ is critical because a value too high will lead to underfitting, while a value too low will not sufficiently regularize the parameters.
- Regularization techniques are integrated into the cost function for penalizing the magnitude of model parameters W
- For managing model complexity and preventing overfitting/Underfitting.

DCGANs Results



DCGANs Results



Figure 10: More face generations from our Face DCGAN.

Guidelines for DCGANs

- Architecture guidelines for stable Deep Convolutional GANs
 - Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
 - Use Batchnorm in both the generator and the discriminator.
 - Remove fully connected hidden layers for deeper architectures.
 - Use ReLU activation in generator for all layers except for the output, which uses Tanh.
 - Use LeakyReLU activation in the discriminator for all layers.

Research Tasks

- What is momentum? Explain the use of the Momentum term in Neural Networks.

Thank You 😊